

Labor VI (E): Abgabe (45 min, Teil 1: 12 Punkte, Teil 2: 6 Punkte)

- Lösen Sie die untenstehenden Aufgaben und laden Sie Ihre Quelldateien `task.cpp` und `task.c` nach TUWEL hoch.
- Sie können Ihre Programme mittels folgender Zeile kompilieren und ausführen:

```
g++ -g -std=c++20 -fsanitize=address task.cpp -o taskcpp && ./taskcpp
gcc -g -std=c11 -fsanitize=address task.c -o taskc -lm && ./taskc
```

Aufgabenstellung Teil 1 (Sprache C++, `task.cpp`)

Gegeben sind Nutzerdaten `User` bestehend aus einer Nutzernummer `uid`, Email-Adresse `email` und Nutzernamen `name`:

```
struct User {
    unsigned int uid;
    std::string email;
    std::string name;
};
```

Ebenso vorgegeben ist die Nutzung einer Funktion `validate`, die jedem Eintrag in einer Sequenz von Nutzerdaten entweder den Wert `true` (gültig) oder `false` (ungültig) zuordnet.

```
int main() {
    std::vector<User> users = {
        {1, "e123@cpp.at", "e123"}, // valid
        {2, "e124@cpp.at", ""},     // empty name
        {3, "e1245cpp.at", "e1245"}, // invalid email
        {2, "e125@cpp.at", "e125"}  // duplicate uid
    };
    auto res = validate(users);
    std::vector<bool> expected = {true, false, false, false};
    assert(res == expected);
    return 0;
}
```

Implementieren Sie die Funktion `validate` so, dass die oben vorgegebene Nutzung erfolgreich kompiliert und alle Annahmen (`assert`) erfüllt, wobei ein Eintrag in einer Sequenz von Nutzerdaten gültig ist wenn jede der drei folgenden Bedingungen erfüllt ist:

1. Die Nutzernummer ist eindeutig in der gesamten Sequenz.
2. Die Email-Adresse enthält mindestens einmal das Zeichen '@'.
3. Der Nutzernamen ist nicht leer.

Aufgabenstellung Teil 2 (Sprache C, `task.c`)

Setzen Sie die Funktionalität aus Teil 1 nun in der Sprache C um:

- **main:**
 - Nutzen Sie statt `std::vector` Felder fester Größe.
 - Verwenden Sie äquivalente `assert`-Aufrufe.
- **validate:**
 - Sowohl das Feld mit Nutzerdaten, als auch das Feld für die Ergebnisse werden übergeben.